

BACK_END (SUNDAY 22/02/2026) :

CLEANUP_OLD_data.py : this code in backend is used to clean up and refresh up our site web and our data base and with it all the old and unnecessary data is being removed by a simple method where our code has a cutoff which is the number off days from todays to the day where this information is being published and the max is 90 days and by this code :

- Date calculation : creates a cutoff date and anything older than this gets deleted
- Delete old trips : any trip with a date before cutoff is removed and every trip marked with completed or cancelled is removed all those trips are counted and removed if not DRY-RUN
- Delete old notifications : same thing finds all old notifications and remove them and those that have been read and same thing counts then deletes
- Delete expired tokens : same procedure
- The dry run is a test where we can look at what will happen to our data base even without making any changes to the data base we can call it a what-if analysis tool

CREATE_INITIAL_data.py : with this code our site web can create essential baseline data that are needs to function properly and it's run once when setting up a new environment and with this command we can create two types of fundamental data:

- User roles : which indicates what's the role of our user it can be a user or an admin or a moderator
- User preferences : which indicates what the user prefers like musique, silence, femur, etc

First it creates a role with user just a regular user who can book or create trips or admin which is a full system administrators with all permissions or a moderator can moderate content but has limited admin access, and by using `get_or_created` so it's simple first it will try to get a role whit that name if it exist so it will get it else it will creates it whit the right description and at the end it returns the object and Boolean which is true or false if created and if it's created then we will track it and only counts and reports newly created items and not the existing ones and by this command we only have to create what's idempotent and return what's already created and it can be used multiple times

LOAD_FUEL_PRICES.py : by using this code and it's Django commands our site web will be able to find and give the prices of every trip based on some information :

- **Json** : with this file our database will be able to find and change the prices of gasoline and diesel and import fuel prices by wilaya from it
- **Command arguments** : the user must provide the path to the JSON file where this file can find and deduce the fuel price and the trip price
- **File reading** : in this case the file JSON will be opened and read the prices and by having the date by using the `timezone.now().data()` and by this the prices in the file will be changing day per day every day
- **Update_or_create()** : by filling the records of wilaya, fuel type, effective date the update price will be shown if and only if this record is already exists else a new record will be created and used
- **Decimal handling for monetary values** : by this the prices will be converted to decimal type to avoid floating point errors and use `str()` to prevent precision issues during conversion
- **Last check** : This is a multi-level exception handling pattern that catches different types of errors and handles them appropriately

```
except FileNotFoundError:
    self.stdout.write(self.style.ERROR(f"Fichier non trouvé: {json_file}"))
except json.JSONDecodeError:
    self.stdout.write(
        self.style.ERROR(f"Erreur de format JSON dans: {json_file}")
    )
except Exception as e:
    self.stdout.write(self.style.ERROR(f"Erreur: {str(e)}"))
```

UPDATE_USER_STATISTICS.py : with this Django command updates statistics of all users by the user id which helps the system to update new user stats for all users in our base and the command `update_statistics()` can update number of trips taken and of trips offered and average rating and total kilometers traveled and member since calculations etc

- By using this part the search for a user id and then if provided it will update only that specific user if omitted it will update all users

```
def handle(self, *args, **options):
    user_id = options.get("user_id")

    if user_id:
        try:
            user = User.objects.get(id=user_id)
            user.update_statistics()
            self.stdout.write(
                self.style.SUCCESS(f"Statistiques mises à jour pour: {user.email}")
            )
        except User.DoesNotExist:
            self.stdout.write(
```

```

        self.style.ERROR(f"Utilisateur avec ID {user_id} non
trouvé")
    )
else:
    users = User.objects.all()
    count = 0

    for user in users:
        user.update_statistics()
        count += 1

    if count % 100 == 0:
        self.stdout.write(f"Traités: {count} utilisateurs...")

self.stdout.write(
    self.style.SUCCESS(
        f"\nStatistiques mises à jour pour {count} utilisateurs!"
    )
)

```

Start Command

|

v

Was --user-id provided?

|

+---+---+

|

|

Yes

No

|

|

v

v

Get ONE

Get ALL

user by

users from

ID

database

|

|

v	v
Update	Loop through
that	ALL users,
ONE user	update each
v	v
Show	Show
single	progress &
user	total
Message	count

SAIDI_MAHMOUD_2CP7